Übungsblatt 8: Multicore-Synchronisation und Safety

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung

Zielsetzung

Drei getrennte Versuche auf dem STM32H755: ein gemeinsamer Zähler mit HSEM, ein Watchdog-Fenster, ein MPU-Zugriff mit bekannter Adresse. M7 bleibt Bare-Metal, M4 bleibt FreeRTOS. Shared Memory und non-cacheable Region wie in Vorlesung 6. Das ist eine Lehrdemonstration, keine freigegebene Sicherheitsarchitektur. Im Kursrepository liegt kein Cube-Projekt. PCLK, LSI-Grenzen, freie HSEM-ID und Reset-Scope tragen Sie aus dem generierten Projekt ein. Leere Messzellen sind keine durchgeführten Tests.

Aufgabe 1: Konfigurationstabelle

Größe	Ihr Projekt
Board, nur falls NUCLEO-H755ZI-Q	
HAL- und Cube-Version	
M7-Takt, M4-Takt	
PCLK von WWDG1 (APB3) oder WWDG2 (APB1)	
RTOS-Tick	
SRAM4-Adresse und MPU-Attribute	
Boot-HSEM, meist 0, nicht für den Zähler	
Zähler-HSEM, Vorschlag 1, wenn frei	
IWDG-Instanz und LSI-Min/Max	

`HAL_HSEM_Take` und `HAL_HSEM_FastTake` im Header nachlesen. Process-ID ist Ihre Konstante, nicht die Task-Handle. Ob ein zweiter Take desselben Besitzers erfolgreich ist, notieren Sie aus dem Treiber, nicht aus dieser Anleitung.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 1

Ohne PCLK keine Millisekunden für das WWDG-Fenster. Lehrannahme: Prescaler 8, PCLK 100 MHz ergibt 0,32768 ms je Tick. Bei 50 MHz sind es 0,65536 ms. 200 MHz ist kein zulässiger APB-Takt. IWDG-Nennwert $(500 + 1) \times 32/32000 = 0{,}501\,\text{s}$ nur bei 32 kHz. Die LSI-Grenzen spreizen das Band.

Aufgabe 2: Endlicher Zähler

`shared_count` einmal auf 0 setzen, bevor beide Kerne die Schleife beginnen. Je Kern genau N Versuche. Danach erst lesen. Erwartung mit Schloss: $2N$, wenn jeder Take geklappt hat. Fehlversuche extra zählen, sonst ist $2N$ die falsche Erwartung.

M7 ruft kein `osDelay`. M4 gibt auch nach erfolgreichem Take die CPU ab. Unter dem Schloss nur die Addition und die Barrieren.

Ungeschützter Vergleich: dieselbe Schleife ohne HSEM. Ein Zufallstreffer $2N$ beweist keine Race-Freiheit. Mehrere M4-Tasks an derselben ID brauchen zusätzlich einen lokalen Mutex. FastTake nicht dafür verwenden.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 2

```
/* M7, außerhalb des Schlosses warten */
for (uint32_t i = 0; i < N; i++) {
    if (HAL_HSEM_Take(1, 0x37u) == HAL_OK) {
        __DMB();
        shared_count += 1u;
        __DMB();
        HAL_HSEM_Release(1, 0x37u);
        hits_m7++;
    }
    HAL_Delay(1);
}
```

```
/* M4 */
for (uint32_t i = 0; i < N; i++) {
    if (HAL_HSEM_Take(1, 0x34u) == HAL_OK) {
        __DMB();
        shared_count += 1u;
        __DMB();
        HAL_HSEM_Release(1, 0x34u);
        hits_m4++;
```

```
    }
    osDelay(1);
}
```

Soll: `shared_count == hits_m7 + hits_m4`. Bei N Erfolgen je Kern ist das $2N$.

Warten ohne verlorenes Aufwecken, eigener Versuch: Benachrichtigung scharf schalten, Take, bei Fehler auf ein Thread-Flag warten, das die ISR setzt, danach Take erneut. Flag ist kein Besitz. Timeout beendet den Versuch. NVIC-Priorität wie in Vorlesung 5. Ob `HAL_HSEM_IRQHandler` die Benachrichtigung löscht, im Treiber prüfen und danach neu aktivieren.

Aufgabe 3: Watchdog, ein Fall nach dem anderen

Instanz wählen, Clock an, Prescaler, Fenster, Reload, EWI, NVIC, Handler der Instanz, `HAL_WWDG_IRQHandler`, Callback. Resetflags lesen, bevor sie gelöscht werden. Debug-Freeze aufschreiben. Messung mit und ohne Debugger unterscheiden.

Zuerst die Tickzeit aus Ihrem PCLK rechnen. Test-Delays mit Abstand zur frühen und zur späten Grenze. `osDelay(15)` aus älteren Texten nicht übernehmen.

Reihenfolge, nie zwei Fälle zugleich: früher Refresh, gültiger Refresh, kein Refresh bis EWI, CPU-Hog ohne fremdes Füttern, blockierte Tasks mit Idle-Hook. GPIO-Flanke für das Fenster. Die LED zeigt einen kurzen EWI-Puls schlecht.

EWI: kein Flash, kein RTOS-Aufruf bei unerlaubter Priorität. Ein RAM-Satz nur, wenn er den Reset wirklich überlebt. Sonst weglassen. Policy entweder „nicht mehr füttern“ oder „bewusst noch einmal füttern“, nicht beides.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 3

Zu früh kann ohne EWI resetten. Bei `0x40` kann ein erlaubtes Refresh den Reset noch verhindern. Der Zähler läuft für diesen Reset nicht bis 0, sondern vom Übergang `0x40` nach `0x3F`. Ob Gleichheit mit dem Fensterwert noch legal ist, im RM0399 nachsehen und nicht als einzigen Testpunkt nutzen. IWDG-Fenster nur behaupten, wenn das Register in Ihrer Init wirklich aktiv ist.

Aufgabe 4: MPU und Tabelle

Eine dokumentierte Adresse in einer verbotenen Region schreiben. Das Disassembly muss diesen Zugriff enthalten. `*NULL` ist kein sauberer Test. Adresse 0 kann auf diesem Chip abgebildet sein.

Erwartung: MemManage oder HardFault, je nach Enable-Bits. Kein „nur die Task stirbt“. Der gewöhnliche CM4-Port legt keine Guard je Task an.

Experiment	analytische Erwartung	beobachtet
Zähler ohne Schloss	nicht zuverlässig $2N$	offen
Zähler mit HSEM	Summe der Treffer	offen
Freigabe vor dem Warten	Task kann hängen	offen
WWDG früh / gültig / spät	aus Ihrer PCLK-Tabelle	offen
Idle füttert, Tasks blockiert	kein Reset	offen
MPU, verbotene Adresse	Fault	offen

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 4

ReleaseAll nicht benutzen, um nach einem stillen Partner einfach weiterzuzählen. Ohne sicheren Halt des Partners und ohne neue Initialisierung ist der Lehrweg ein gemeinsamer Reset. Ein einzelner Bit in `uint16_t` macht aus `0x0064` zum Beispiel `0x8064`, nicht 65535. Keine Zelle ausfüllen, die Sie nicht gemessen haben. Der Folienbau ist kein Lauf auf dem Board.